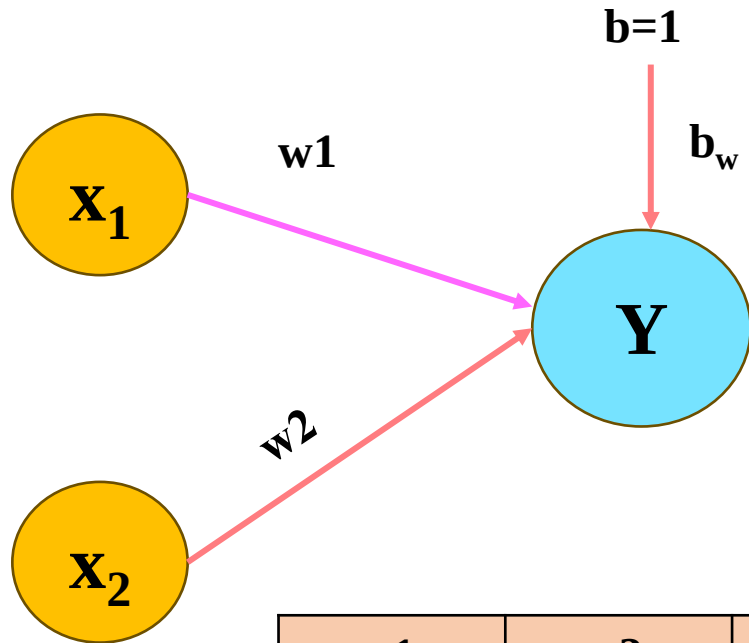


Learning Rules

Learning Rules

Learning	Single weight adjustment	Initial Weights	Learning	Neuron Characteristics	Neuron/ Layer
Hebbian	$c * f(\text{net}_i) * X_i$	0	Unsupervised	Any	Single neuron
Perceptron	$c * (d_i - f(\text{net}_i)) * X_i$	Any	Supervised	Discrete	Single neuron
Delta	$c * (d - f(\text{net}_i)) * f'(\text{net}_i) * X_i$	Any	Supervised	Continuous	Single neuron
Widrow-Hoff	$c * (d_i - \text{net}_i) * X_i$	Any	Supervised	Any	Single neuron

Design a Hebb Network for implementing Logical AND gate considering bipolar inputs and target.

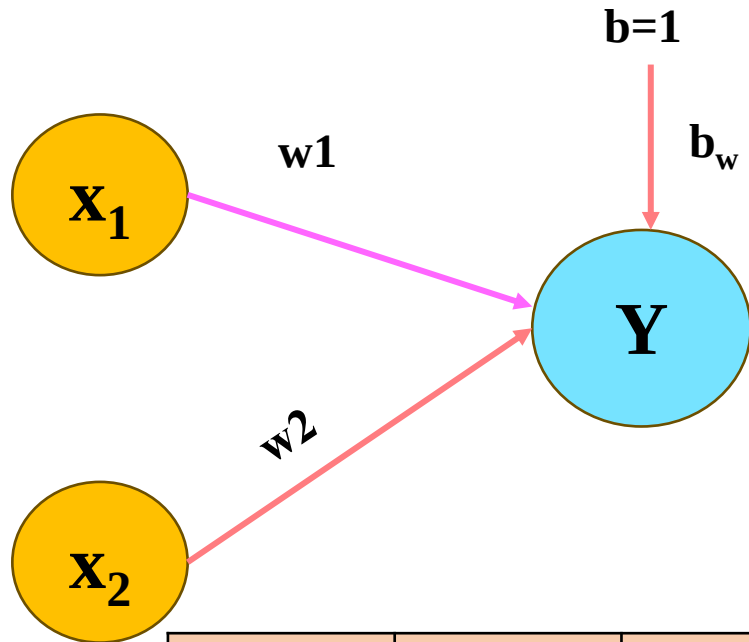


$$Y = x1 * w1 + x2 * w2 + b$$

$$w_1 = w_2 = b_w = 0$$

x1	x2	Output
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

Design a Hebb Network for implementing Logical AND gate considering bipolar inputs and target.



$$Y = x_1 * w_1 + x_2 * w_2 + b$$

$$w_1 = 2 \quad w_2 = 2 \quad b_w = -2$$

x1	x2	Output
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

$$Y = (-1 * 2) + (-1 * 2) + (1 * -2) = -6 < 0 \quad -1$$

$$Y = (-1 * 2) + (1 * 2) + (1 * -2) = -2 < 0 \quad -1$$

$$Y = (1 * 2) + (-1 * 2) + (1 * -2) = -2 < 0 \quad -1$$

$$Y = (1 * 2) + (1 * 2) + (1 * -2) = 2 > 0 \quad 1$$

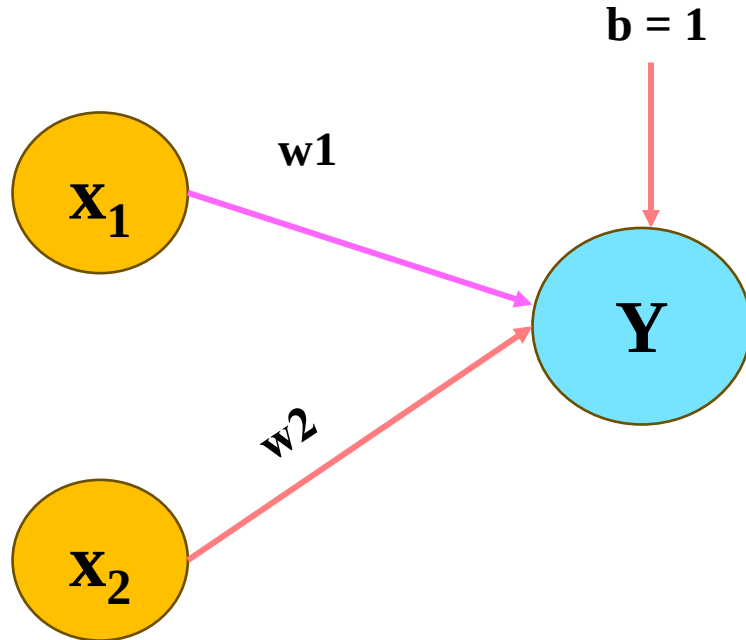
Hebb Rule Numerical 2

- Implement the Hebbian training rule for a network using $f(\text{net}) = \text{sgn}(\text{net})$. The three input vectors for training and the initial weight vector is given as follows:

$$X_1 = \begin{bmatrix} 1 \\ -2 \\ 1.5 \\ 0 \end{bmatrix}, X_2 = \begin{bmatrix} 1 \\ -0.5 \\ -2 \\ -1.5 \end{bmatrix}, X_3 = \begin{bmatrix} 0 \\ 1 \\ -1 \\ 1.5 \end{bmatrix}, \quad w^1 = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{bmatrix}$$

Design a Perceptron Network for implementing Logical AND gate considering bipolar inputs and target.

**Consider Theta as 0
and Learning Rate = 1**



$$Y = x_1 * w_1 + x_2 * w_2 + b$$

$$\begin{aligned} f(x) &= 1 > 0 \\ &= 0 = 0 \\ &= -1 < 0 \end{aligned}$$

x1	x2	Output
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

Perceptron Learning Rule Numerical 2

Set of input vectors & initial weights are given as follows

$$X_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, X_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ 1 \end{bmatrix}, X_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{bmatrix}$$

Learning constant, $c=0.1$. Teachers desired responses for X_1 , X_2 & X_3 are $d_1=-1, d_2=-1$ & $d_3=1$.

Implement and show the steps of perceptron learning rule

Delta Learning Rule

$$\Delta w_{ij} = \eta(d_i - o_i)f'(\text{net}_i)x_j, \quad \text{for } j = 1, 2, \dots, n$$

- The delta learning rule is only valid for continuous activation functions and in the supervised training mode. The learning signal for this rule is called delta.

- Gradient defined as:

$$E \triangleq \frac{1}{2}(d_i - o_i)^2$$

- It also can be called the continuous perceptron training rule.
- The delta learning rule can be generalized for multilayer networks.

Example: Delta Learning Rule

- **Perform 02 training stage using DLR.** Consider 3 input vectors x_1, x_2, x_3

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}$$

- Learning Constant $c = 0.1$ $\text{Lambda} = 1$ & initial weights

$$\mathbf{w}^1 = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{bmatrix}$$

- Desired Output $d_1 = -1, d_2 = -1, \text{ and } d_3 = 1$

- Activation Function : **Bipolar continuous activation function**

Step 1: For input vector x_1

EPOCH = 01

- Calculate $\text{net}^1 = w_1^T * x_1$

- $\text{net}^1 = [\textcolor{red}{1} \textcolor{green}{-1} \textcolor{blue}{0} \ 0.5] * \begin{bmatrix} \textcolor{red}{1} \\ \textcolor{green}{-2} \\ \textcolor{blue}{0} \\ -1 \end{bmatrix} = [(\textcolor{red}{1} * \textcolor{red}{1}) + (\textcolor{green}{-1} * \textcolor{green}{-2}) + (\textcolor{blue}{0} * \textcolor{blue}{0}) + (0.5 * -1)]$

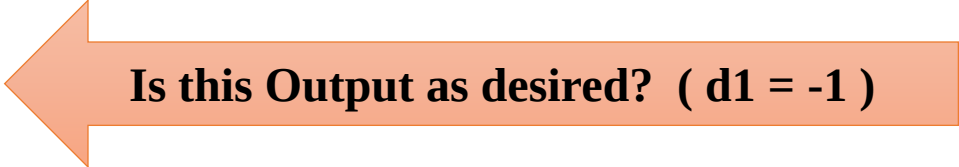
- $\text{net}^1 = [1 + 2 + 0 - 0.5]$

- $\text{net}^1 = 2.5$

Step 1: For input vector x_1

- Calculate Output $O^1 = \frac{2}{1 + e^{-(1*2.5)}} - 1$

- $O^1 = 0.8482$



Is this Output as desired? ($d1 = -1$)

Step 1: For input vector x_1

- Calculate Output $O^1 = \frac{2}{1 + e^{-(1*2.5)}} - 1$

- $O^1 = 0.8482$

Is this Output as desired? ($d1 = -1$)

- Next compute Derivative for Bipolar Continuous Activation Function

$$f'(\text{net}) = \frac{1}{2}(1 - o^2)$$

$$f'(\text{net}^1) = \frac{1}{2} (1 - (0.8482)^2)$$

$$f'(\text{net}^1) = 0.1402$$

Step 1: For input vector $\mathbf{x}_1$

$$\mathbf{w}^2 = c(d_1 - o^1)f'(net^1)\mathbf{x}_1 + \mathbf{w}^1$$

- Update the weights according to Delta Learning Rule

$$\bullet W^2 = 0.1 * ((-1) - 0.8482) * 0.1402 * \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{bmatrix}$$

$$\bullet W^2 = (-0.02591) * \begin{bmatrix} 1 \\ -2 \\ 0 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{bmatrix}$$

$$\bullet W^2 = \begin{bmatrix} 0.9740 \\ -0.9481 \\ 0 \\ 0.5259 \end{bmatrix}$$

$$W^2 = [0.9740 \quad -0.9481 \quad 0 \quad 0.5259]^T$$

Step 2: For input vector x_2

- Calculate $net^2 = w_2^T * x_2$

- $net^2 = [0.9740 \quad -0.9481 \quad 0 \quad 0.5259] * \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix}$

$$net^2 = -1.9481$$

- Calculate O^2 $O^2 = -0.7505$

- Calculate derivative $f'(net^2)$ $f'(net^1) = \frac{1}{2} (1 - (-0.7505)^2)$
 $f'(net^2) = 0.2184$

Step 2: For input vector x_2

- Update the weights according to Delta Learning Rule

$$\begin{array}{ccccccc}
 & & & & x_2 & & w_2 \\
 & d_2 & o_2 & f'net_2 & & & \\
 \bullet \ W^3 = & 0.1 & * & ((-1) - (-0.7505)) & * & 0.2184 & * \begin{bmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{bmatrix} + \begin{bmatrix} 0.9740 \\ -0.9481 \\ 0 \\ 0.5259 \end{bmatrix}
 \end{array}$$

$$\bullet \ W^3 = \begin{bmatrix} 0.9740 \\ -0.9563 \\ 0.0027 \\ 0.5313 \end{bmatrix}$$

Step 3: For input vector x_3

- Calculate $\text{net}^3 = w_3^T * x_3$

- $\text{net}^3 = [0.9740 \quad -0.9563 \quad 0.0027 \quad 0.5313] * \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix}$

$$\text{net}^3 = -2.4603$$

- Calculate O^3
- $$O^3 = -0.8426$$

- Calculate derivative $f'(\text{net}^3)$
- $$f'(\text{net}^1) = \frac{1}{2} (1 - (-0.8426)^2)$$
- $$f'(\text{net}^3) = 0.1450$$

Step 3: For input vector x_3

- Update the weights according to Delta Learning Rule

$$\begin{array}{ccccccc}
 & & & & x_3 & & w_3 \\
 & d_3 & o_3 & f'net_3 & & & \\
 \bullet \ W^4 = & 0.1 & * & ((1) - (-0.8426)) & * & 0.1450 & * \begin{bmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{bmatrix} + \begin{bmatrix} 0.9740 \\ -0.9563 \\ 0.0027 \\ 0.5313 \end{bmatrix}
 \end{array}$$

$$\bullet \ W^4 = \begin{bmatrix} 0.9473 \\ -0.9296 \\ 0.0160 \\ 0.5046 \end{bmatrix}$$

EPOCH = 02

$$W_1 = \begin{bmatrix} 0. \\ 0. \\ 0. \\ 0. \end{bmatrix}$$

$$W_3 = \begin{bmatrix} 0. \\ 0. \\ 0. \\ 0. \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0. \\ 0. \\ 0. \\ 0. \end{bmatrix}$$

$$W_4 = \begin{bmatrix} 0. \\ 0. \\ 0. \\ 0. \end{bmatrix}$$

Widrow – Hoff Learning Rule

$$\Delta w_{ij} = c(d_i - \mathbf{w}_i^t \mathbf{x})x_j, \quad \text{for } j = 1, 2, \dots, n$$

- Used for Supervised training networks
- This rule can be considered a special case of the delta learning rule.
- This rule is sometimes called the LMS (least mean square) learning rule. Weights are initialized at any values in this method.

It is independent of the activation function.

Example: Widrow – Hoff Learning Rule

- **Perform 02 training stage using WH-LR.**

- Consider 2 input vectors x_1, x_2

- $X1 = \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix}$

$$\begin{bmatrix} 1 \\ -2 \\ -1 \end{bmatrix}$$

- Learning Constant $c = 0.25$ $\text{Lambda} = 1$ & initial weights

$$\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

- Desired Output : $d1 = -1$ $d2 = 1$

EPOCH = 01

- Compute O_1

$$O_1 = W_i^T * X_1$$

$$O_1 = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix}$$

$$O_1 = 1$$

- Compute Change in weight

$$\Delta\omega_i = c(d_i - o_i)x_i$$

$$\Delta\omega_1 = 0.25 * ((-1) - 1) * \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix}$$

$$\Delta\omega_1 = \begin{bmatrix} -1 \\ 0 \\ 0.5 \end{bmatrix}$$

- Calculate Update Weight

$$W_2 = W_{1_old} + \Delta\omega_1$$

$$W_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ 0 \\ 0.5 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0 \\ 0 \\ 1.5 \end{bmatrix}$$

EPOCH = 01

- Compute O_2
- $O_2 =$

- Compute Change in weight

$$\Delta\omega_i = c(d_i - o_i)x_i$$

- $\Delta\omega_2 = \begin{bmatrix} - \\ - \\ - \end{bmatrix}$

- Calculate Update Weight

- $w_3 = w_{2_old} + \Delta\omega_2$

$w_3 = \begin{bmatrix} - \\ - \\ - \end{bmatrix}$

Linear Separability

- A data set is linearly separable if there exists a straight line (2d) or hyperplane (higher dimensions) that can separate the data into 2 classes without any misclassifications.

Non-Linear Separability

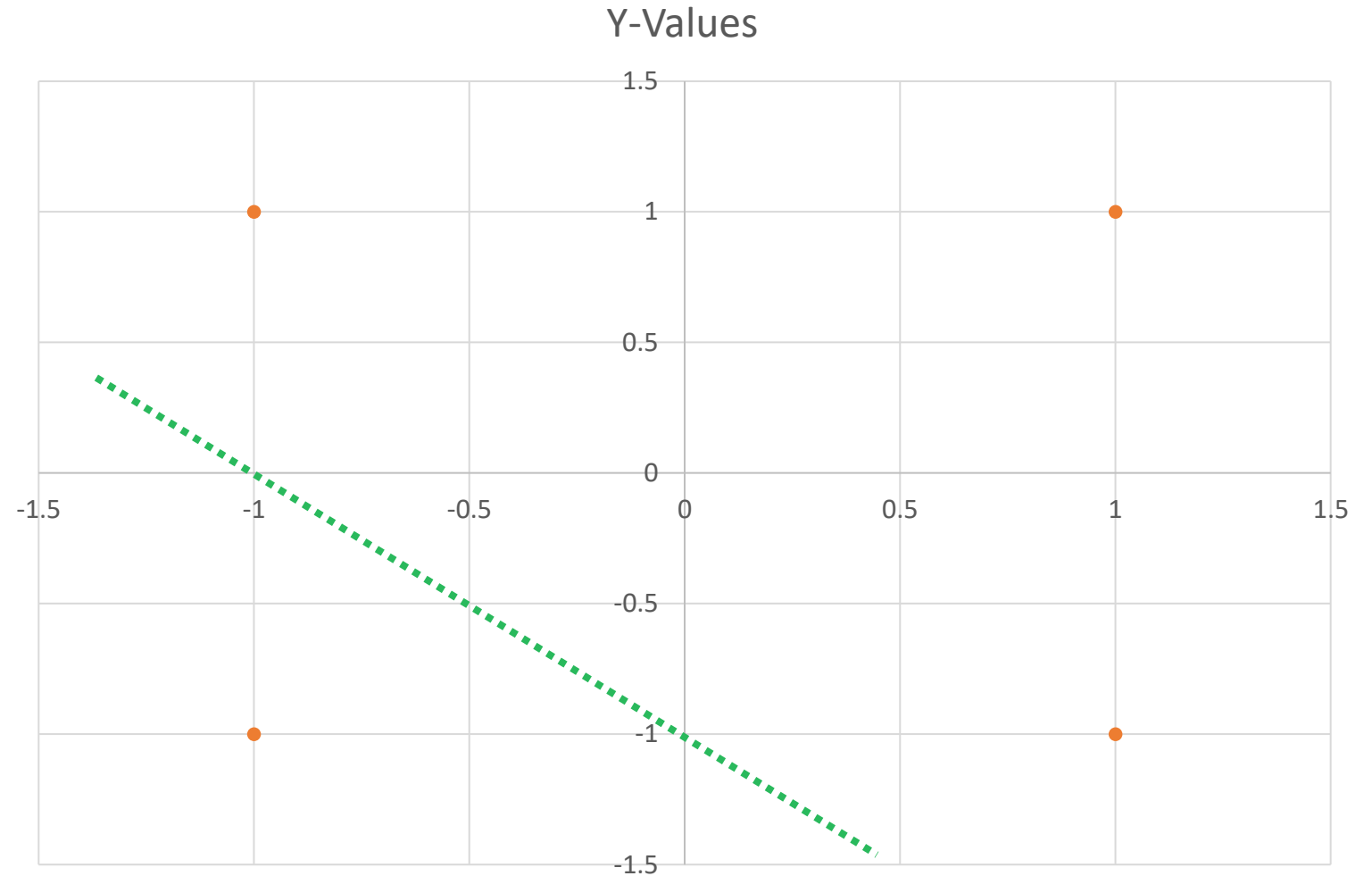
- A dataset is non-linearly separable if no straight line can separate the data perfectly into distinct classes without any errors.
- Such data requires more complex boundaries to be separated accurately.

Single vs Multilayer Network

- A Single Layer Network can learn to classify linearly separable data by adjusting its weights and bias during training.
- But SLN lacks complexity to model complex boundaries for non-linear data.
- So, to model non-linearly separable data we have Multi-layer networks with hidden layers to handle it.

Consider OR Function with Bipolar Inputs

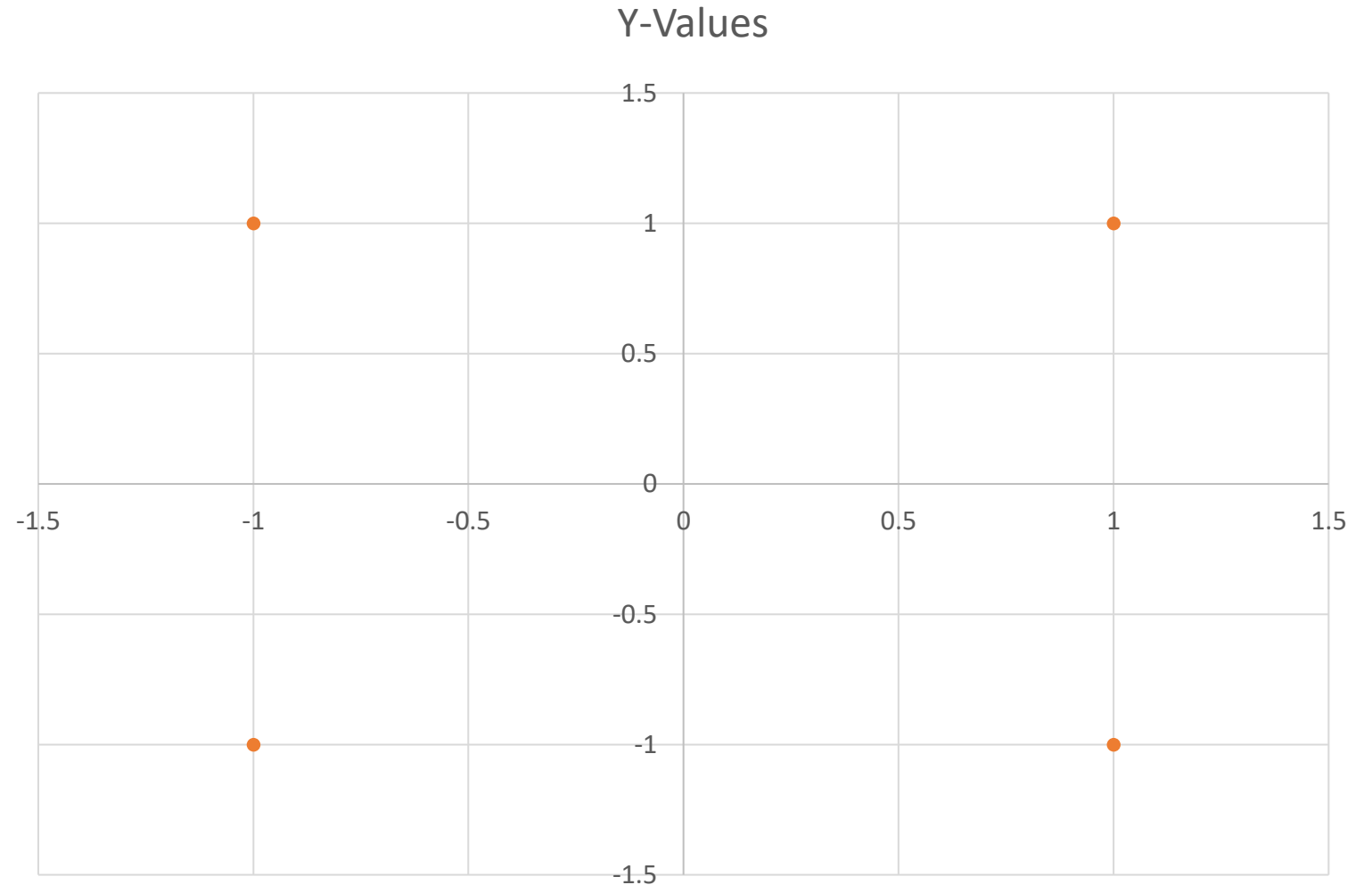
x1	x2	Output
-1	-1	-1
-1	1	1
1	-1	1
1	1	1



Consider XOR Function with Bipolar Inputs

x1	x2	Output
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

Not Linearly Separable Data !!



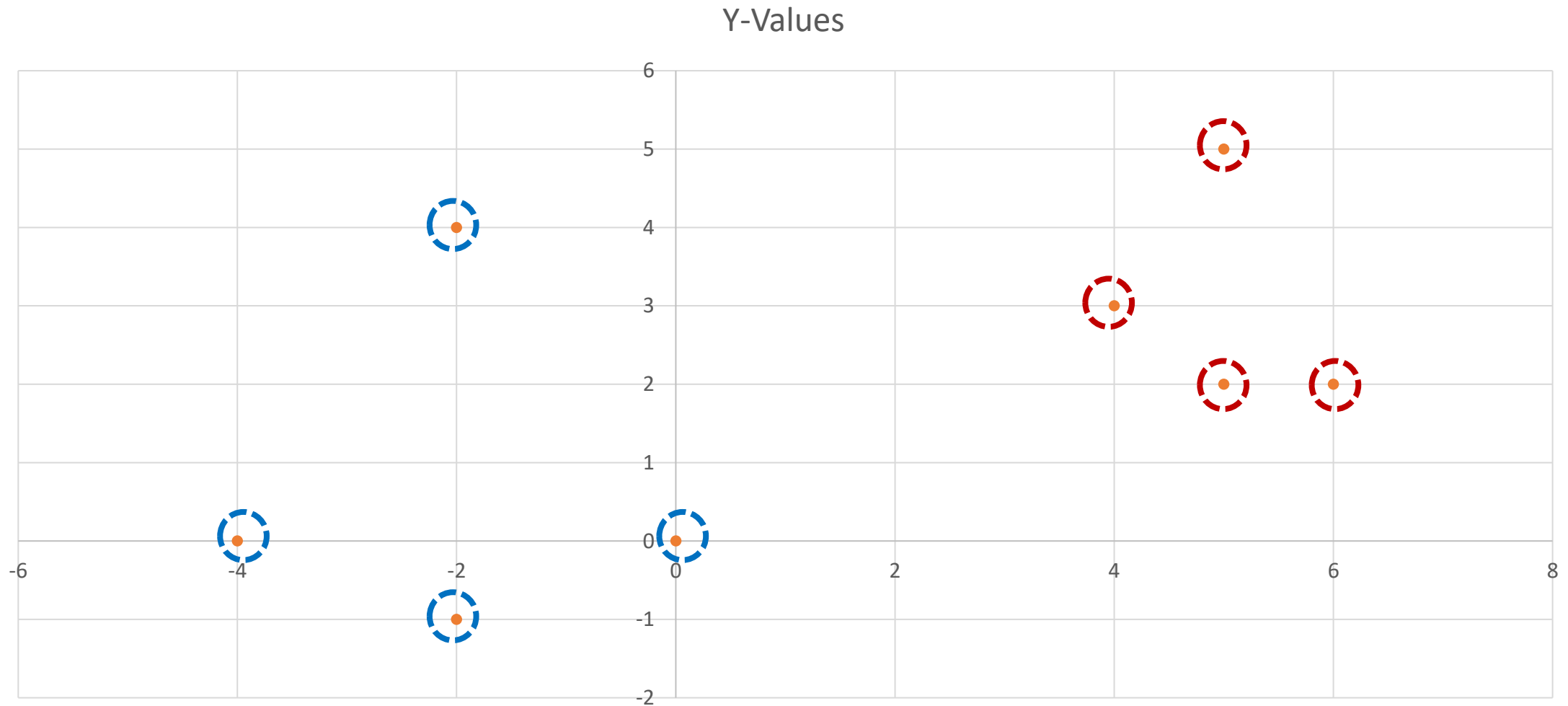
Numerical Example:

The prototype points are given as :

$X_1 = [5, 2]$, $X_2 = [6, 2]$, $X_3 = [4, 3]$, $X_4 = [5, 5] \Rightarrow$ Class I

$X_5 = [0, 0]$, $X_6 = [-2, -1]$, $X_7 = [-2, 4]$, $X_8 = [-4, 0] \Rightarrow$ Class II

i) Find if they are linearly separable.



Line Equation in a 2D Plane $y = mx + c$

- m is the slope of the line,
- c is the y-intercept (the point where the line crosses the y-axis)
- Select any 2 approximate points through which line should pass.
- Point 1 (1, - 0.5) Point 2 (5, 5.5)
- Compute $m = \frac{y_2 - y_1}{x_2 - x_1}$ $m = \frac{5.5 - (-0.5)}{5 - 1}$ $m = 1.5$

- Compute c Substitute one of the point in the 2d line equation
- Consider Point 1 (1 , -0.5) $\rightarrow$ $-0.5 = (1.5) * 1 + c \rightarrow c = -2$
- When $c = -2$, means line will cut through y axis at $y = -2$
- Now to know where it will cut x axis, put $y=0$ in 2d line equation with $m = 1.5$ and $c = -2 \rightarrow x = 1.3$
- Final 2D Line Equation $\rightarrow y = 1.5 x + (- 2)$
- Plot the line on the graph

Numerical Example:

The prototype points are given as :

$X_1 = [5, 2]$, $X_2 = [6, 2]$, $X_3 = [4, 3]$, $X_4 = [5, 5] \Rightarrow$ Class I

$X_5 = [0, 0]$, $X_6 = [-2, -1]$, $X_7 = [-2, 4]$, $X_8 = [-4, 0] \Rightarrow$ Class II

i) Find if they are linearly separable.

